

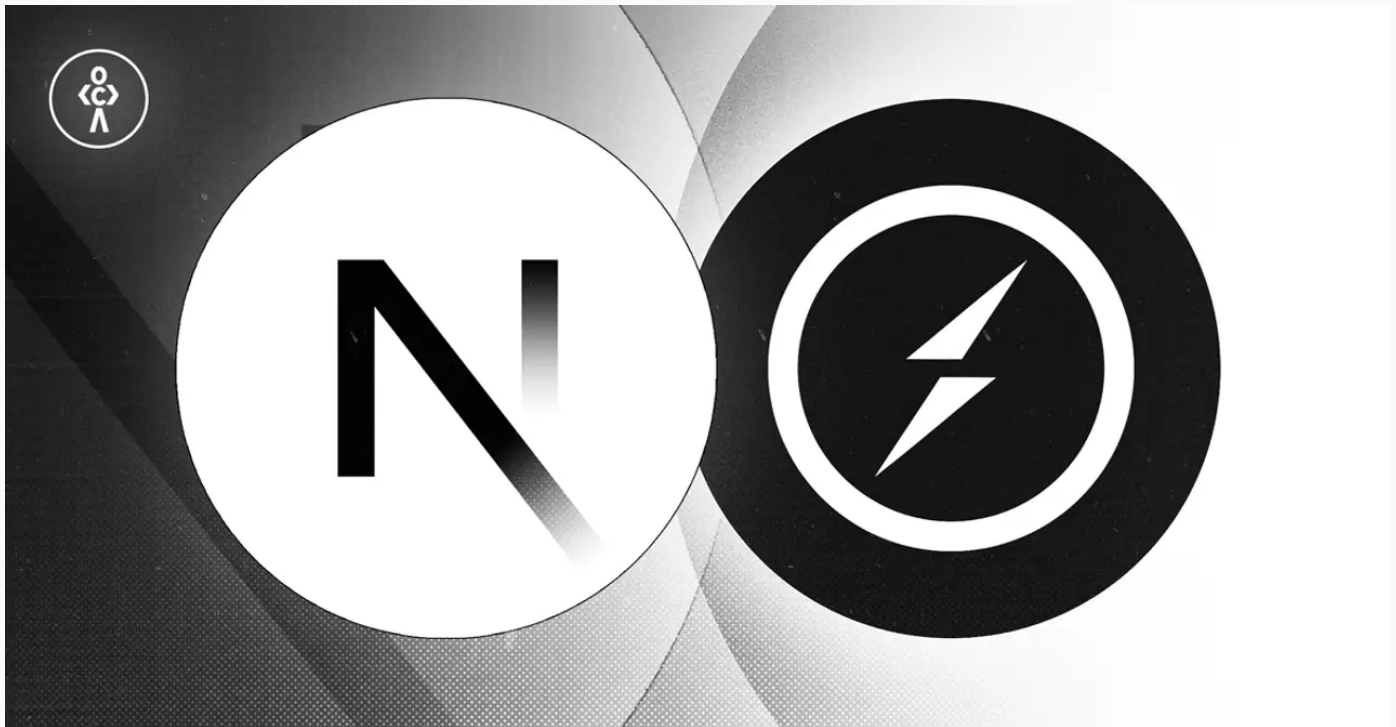


Login

Get Started

Wednesday, Sep 14th 2022

How to use Socket.io with Next.js?



The WebSocket API, which is a full-duplex communication channel over a single TCP connection socket, is used for real-time communication on the web. This allows numerous clients to connect to a server and share real-time data, allowing for a variety of experiences like online games, interactive papers, and chat rooms.

Introduction

[Next.js](#) helps developers build large complex applications with ease by using



down the system. Using microservices in Next.js, one can easily scale a large-scale system and can divide it into different chunks for feature updates. It helps add independent features to the application without changing other services.

The development of Next.js applications can be readily scaled in both horizontal and vertical orientations. Both client-side and server-side apps are built with Next.js. It has an open-source JavaScript runtime environment/model that allows single modules to be cached.

Applications

Both the tech stack are being widely used for developing web apps. Large tech giants like Netflix, Airbnb, and Instagram use this tech stack in their applications.

It is largely used by developers in the backend of an application. Due to its non-blocking I/O and asynchronous nature, it has become the primary language used on the server side. Tech giants like Netflix, PayPal, LinkedIn, and Uber use it for building API and servers.

Why use it:

- Easy to learn as it uses JavaScript
- Used for agile development and prototyping
- Provides fast and scalable services
- Asynchronous nature
- Uses “Single-threaded-event-loop” architecture

So, what exactly is Socket.io?

Socket.io is the most well-known WebSocket wrapper for [Next.js](#). It's a browser-based package that includes a Next.js server and a client library. In this tutorial, I'll demonstrate a WebSocket connection between a server and a Next.js application using Socket.io.

It's worth noting that Vercel's serverless functions don't support WebSockets. Despite the fact that Vercel isn't the only way to deploy a Next.js app, this constraint may be a deal-breaker for some.



For real-time communication on deployments to Vercel's platform, Vercel suggests using third-party solutions. They also provide instructions for integrating Pusher, one of these third-party systems.

We'll begin by utilizing the Create Next App package to create a Next.js application.

Run the `npx create-next-app websocket-example`, in which `WebSocket-example` is the project's name. A boilerplate app is created as a result of this.

Constructing the server

The `pages/api` folder will appear when you navigate inside the project folder. We create the endpoints that the client will use in this section. Any file in the `pages/api` directory is considered as a `/api/* endpoint`.

Make a `socket.js` file in the `pages/api` folder. This file will handle the WebSocket connection by guaranteeing that there is only one WebSocket instance, which we will access via a client request to `/api/socket`.

Next.js comes with its own server by default, so you won't need to configure anything unless you need a special server. We don't need to set up a special server for this simple WebSocket example.

We need to verify in the `socket.js` file if the HTTP server already has a socket connection open. If not, we'll use Socket.io instead.

As you may know, in order for an API route to work, Next.js requires a function with the `req` and `res` parameters corresponding to `http.IncomingMessage` and `http.ServerResponse`, respectively, be exported as default.

Here's an example of an API route.

```
export default function handler(req, res) {}
```

Now we must apply this technique to create a socket connection if one does not already



We check `res.socket.server.io`, which does not exist by default and indicates that the socket connection is not in existence, inside the default exported function. If the `io` object on `res.socket.server` does not exist, we create a new socket connection by instantiating a `Server` and passing it to the `res.socket.server` as an input.

The resulting object is then put to `res.socket.server.io` so that when a new request comes in after the instantiation, `res.socket.server.io` is not undefined.

We just finish the request outside of the `if` statement to prevent it from becoming stalled.

```
import { Server } from 'Socket.IO'

const SocketHandler = (req, res) => {
  if (res.socket.server.io) {
    console.log('Socket is already running')
  } else {
    console.log('Socket is initializing')
    const io = new Server(res.socket.server)
    res.socket.server.io = io
  }
  res.end()
}

export default SocketHandler
```

On the client-side, using the endpoint

We've developed an endpoint to start the WebSocket connection so far. On the client-side, we must now use this endpoint.

The `index.js` file, which is the route for the main directory, may be found in the `/pages` directory. We're going to create a component with an input field in this file. We'll then send the changes to the server, which will be broadcast to all other connected clients. This is a simple demonstration of real-time communication.

The socket connection is the first thing we need to accomplish in the component. We'll implement this by using the `useEffect` Hook with an empty array dependency, which is essentially the same as the `componentDidMount` lifecycle Hook



Because we need to make a call to the `/api/socket` endpoint to open the socket connection, `SocketInitializer` is an async function.

After this call is resolved, we initialize the `io()` object and assign it to a variable outside the function scope, ensuring that it remains constant throughout the component's lifecycle.

Then, to confirm that the client is connected, we use `socket.on()` to wait for a connect signal and log a message. This is a reserved event that should not be manually emitted. The list of reserved events can be seen here.

With a `[]` dependence, we must now call this function from within the `useEffect` Hook. Because effect callbacks should be synchronous to avoid race situations, we wrap the `socketInitializer` inside another method.

```
import { useEffect } from 'react'
import io from 'Socket.IO-client'
let socket

const Home = () => {
  useEffect(() => socketInitializer(), [])

  const socketInitializer = async () => {
    await fetch('/api/socket')
    socket = io()

    socket.on('connect', () => {
      console.log('connected')
    })
  }

  return null
}

export default Home;
```

You'll see the "Connected" message written in the console if you visit the port on your local host, which runs the development server.



After the first client connects, you'll notice "Socket is initializing" logged in the terminal screen where you started the development server.

The "Socket is already running" notice will be printed out if you connect another client by visiting the development server on a different browser tab or screen.

Using Socket.io to send messages

We can now send messages between clients and the server because we've established a socket connection.

We'll start by making a controlled input field. For the onChange event of the input field, all we need is a state and a handler function.

```
const [input, setInput] = useState('')

const onChangeHandler = (e) => {
  setInput(e.target.value)
  socket.emit('input-change', e.target.value)
}

return (
  <input
    placeholder="Type something"
    value={input}
    onChange={onChangeHandler}
  />
)
```

We set the input state to a new value inside onChangeHandler and then send this change to the server as an event. The first socket parameter. The first parameter is the message, and the second is the event name, which is input-change. This is the value of the input field in our example.



We ensure that the socket connection is made by registering a connection listener. The `socket.on()` function, which takes an event name and a callback function as parameters, is then used to subscribe for the `input-change` event within the callback function.

```
io.on('connection', socket => {  
  socket.on('input-change', msg => {  
    socket.broadcast.emit('update-input', msg)  
  })  
})
```

100+ Courses, 500+ Coding Problems - Learn with AI + Practice [3 day free-trial] [Learn More](#) → X

Then we use `socket.broadcast.emit()` inside the callback method, which sends a message to all clients except the one that emitted the `input-change` event. As a result, the `msg`, which is the text input value sent by one of the clients, is only received by other clients.

The `socket.js` file in its final form is as follows.

```
import { Server } from 'Socket.IO'  
  
const SocketHandler = (req, res) => {  
  if (res.socket.server.io) {  
    console.log('Socket is already running')  
  } else {  
    console.log('Socket is initializing')  
    const io = new Server(res.socket.server)  
    res.socket.server.io = io  
  
    io.on('connection', socket => {  
      socket.on('input-change', msg => {  
        socket.broadcast.emit('update-input', msg)  
      })  
    })  
  }  
  res.end()  
}  
  
export default SocketHandler
```



Now, let's return to the client. This time, we must deal with the update-input broadcasted event. To update the input value with the value supplied from the server, just subscribe to the event with `socket.on()` and call the `setInput` function inside the callback.

```
import { useEffect, useState } from 'react'
import io from 'Socket.IO-client'
let socket;

const Home = () => {
  const [input, setInput] = useState('')

  useEffect(() => socketInitializer(), [])

  const socketInitializer = async () => {
    await fetch('/api/socket');
    socket = io()

    socket.on('connect', () => {
      console.log('connected')
    })

    socket.on('update-input', msg => {
      setInput(msg)
    })
  }

  const onChangeHandler = (e) => {
    setInput(e.target.value)
    socket.emit('input-change', e.target.value)
  }

  return (
    <input
      placeholder="Type something"
      value={input}
      onChange={onChangeHandler}
    />
  )
}
```




```
)  
}  
  
export default Home;
```

Conclusion

In this post, we learned how to use Socket.io with [Next.js](#) to create a WebSocket connection that allows the client and server to transfer information in real time.

We also defined an endpoint in the [Next.js](#) backend, which we used to initiate the socket connection in the client. The socket connection was then used to handle the rest of the communication between the server and the client.

Become The Best Next Developer

Codedamn is the best place to become a proficient developer. Get access to hundreds of practice **Next.js** courses, labs, and become employable full-stack Next web developer.

- ✓ Free money-back guarantee
- ✓ Unlimited access to all platform courses
- ✓ 100's of practice projects included
- ✓ ChatGPT Based Instant AI Help (Jarvis)
- ✓ Structured React.js/Next.js Full-Stack Roadmap To Get A Job
- ✓ Exclusive community for events, workshops

[Start Learning](#)

Sharing is caring

Did you like what mansi wrote? Thank them for their work by sharing it on social media.

Share 





Add your comment. Some markdown is supported.



Learn to code interactively - without ever leaving your browser.



SOLUTIONS

[Learning paths](#)

[Explore courses](#)

[Build projects](#)

[Boot playgrounds](#)

[Teach on codedamn](#)

RESOURCES

[Pricing](#)

[Student account](#)

[Developer blog](#)

[Refer and earn](#)

COMPANY

[About](#)

[Careers](#)

[Support](#)

[Terms](#)

[Privacy](#)

[Feedback](#)

POPULAR

[How much JS before React?](#)

[Solidity require keyword](#)

[Next.js on Next.js](#)



[Affiliate program](#)[React.js online compiler](#)[Online compilers](#)[Python online IDE](#)[Online web tools](#)[Next.js best practices](#)[Choose your career](#)[Payable function solidity](#)[Solidity online compiler](#)

© codedamn™ 2023 | All rights reserved.

